Lab3 - Testbench Blocks

The goal of this lab is to create a testbench that checks the functionality of a register file (which could supply values to the 4-bit ALU you worked on in the previous lab). This week, you're expected to develop a modular testbench, including all components discussed in theory class—such as interfaces, generator, driver, monitor, scoreboard or checker, etc.

1. Get Ready for this Lab

In the web of the course at https://docencia.ac.upc.edu/master/MIRI/PV/labs.html, for session Lab3, you will find a compressed package regfile.tgz with the following files:

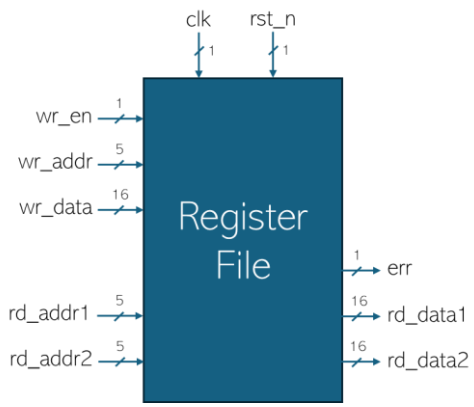
```
$ tar xvfz regfile.tgz
tb_regfile.sv
modelsim.ini
work/
```

As last week, the design is shown as a black box, with only a specification description provided. The work/ directory contains a QuestaSim library with 1+3 RTL design versions, already compiled with vlog:

- regfile_0.v: This design is correct and is expected to successfully pass all tests.
- regfile_1.v – regfile_3.v: In this lab, all three of these alternative designs are known to be incorrect. Each contains bugs, but the exact number is unknown. It is up to the student to identify which features from the specification have been implemented incorrectly. There may be one or multiple faulty features in each design.

The compressed package also contains an empty tb_regfile.sv file for you to write a testbench. The goal of this session is to read carefully the specifications (in the next section, it is supposed to be the same for the design and the verification engineer), create a verification plan, and write a testbench to determine what features are buggy in the alternative designs.

See a diagram of the DUT below:



2. Design Specification

2.1. Overview

The device under test (DUT) is a Register File consisting of 32 entries, each 16 bits wide. It features a single write port, which includes an address input, a data input, and an enable signal. Additionally, there are two read ports, each with its own address input and data output. Writing to the register file occurs on the clock's rising edge, provided that the write enable signal is asserted. Reading from the register file is combinational, meaning that the output data from the read ports instantly reflects the contents stored at the specified addresses. The design also incorporates an active-low synchronous reset, which, when activated, clears all registers to zero.

2.2. Functional Description

The Register File operates according to the following functional behaviors:

- On reset, all registers are cleared to 0.
- On rising edge of clk, if wr_en = 1, the register at wr_addr is updated with wr_data.
- Read ports (rd_data1/2) always reflect the contents of registers at rd_addr1/2 (combinational).
- Input addresses must be stable within a clock cycle
- X outputs are for simulation purposes only

2.3. Interface Definition

Following table describes the specification for the input/output signals:

Signal	Width	Direction	Description
clk	1	Input	Clock signal
rst_n	1	Input	Active-low synchronous reset
wr_en	1	Input	Write enable
wr_addr	5	Input	Write address (0-31)
wr_data	16	Input	Write data
rd_addr1	5	Input	Read port 1 address
rd_addr2	5	Input	Read port 2 address
rd_data1	16	Output	Read port 1 data
rd_data2	16	Output	Read port 2 data
err	1	Output	Error flag for illegal conditions (see below)

2.4. Timing Diagram

The instructions below illustrate the key operational behaviors of the module, detailing how reset, write, and read actions are triggered and reflected in the system:

- Reset: When rst_n = 0, all registers cleared on next rising edge.
    - During reset, err is cleared to 0 on the rising edge
- Write: Occurs on rising edge if wr_en = 1.
- Read: Outputs update immediately when rd_addr changes.
    - Read-during-write to the same address returns the value stored prior to the write (read-first behavior).

2.5. Illegal Conditions

Finally, this section outlines the specific scenarios in which the module identifies illegal operations and asserts the error flag (err = 1):

- Conditions for err = 1:
  - rd_addr1 == rd_addr2
  - wr_en && (wr_addr == rd_addr1 || wr_addr == rd_addr2)
- If an illegal condition is detected combinationally, the DUT does not perform the write on the current rising edge, outputs X on both read ports immediately, and asserts err on the following clock edge.
- err is registered and reflects the illegal condition detected on the previous clock edge.

3. Recommended Approach

Here are the steps you need to take to complete this lab effectively:

A. Prepare a verification plan

Take time to thoroughly review the design specifications and clearly define the requirements and features that need verification. This process will guide you in preparing a list of tests for your testbench and help you determine the most effective testing strategy.

B. Write your testbench

If it helps you, you can follow as a reference, the testbench we described during the lecture W3 - Verification Planning and Testbenches

- Directed Tests:
  - A small set (≈ 5-10) to check basic functionality:
    - Reset clears all registers.
    - Simple write/read sequences.
    - Error conditions (same read address, write/read conflict).
- Randomized Tests:
  - After directed tests, move to constrained random:
    - Generate ~500–1000 randomized transactions.
    - Apply constraints.
    - This ensures variety without exhaustive enumeration.
- All tests?
  - 2^1 (wr_en) x 2^5 x 2^{16} x 2^5 x 2^5 = 4,294,967,296 combinations!!!

After completing your testbench as tb_regfile.sv (recall that each design module can be instantiated by its module name), you can load and compile your testbench and design as:

```
QuestaSim> vmap work ./work
QuestaSim> vdir
# Library vendor : Model Technology
# Maximum unnamed designs : 3
# MODULE regfile_v0
# MODULE regfile_v1
# MODULE regfile_v2
# MODULE regfile_v3
QuestaSim> vlog +acc tb_regfile.sv
QuestaSim> vsim work.tb_regfile
QuestaSim> run -all
```

If you prefer, you can perform the build process from command line using a Makefile.

C. Try your testbench with every provided design (just change the module name in the instantiation)

4. What to Turn In

Please provide a hard copy PDF containing the following items:

1. Please indicate how many hours you spent on this lab. This will not affect your grade, but will be helpful for calibrating the workload for the future.
2. (2 pts) Your verification plan
3. (4 pts) Your testbench tb_regfile.sv code.
    a. How many different tests did you write? (explain)
    b. How much time does it take to run?
4. (2 pts) A table indicating the list of incorrect specification features in each of the alternative designs.
5. (2 pts) This is for you to research and explain in your own words:
   What are clocking blocks in System Verilog and how are they used for verification?

Also include as a separate file/s your testbench source code/s.